

Tuples are ordered sequences of items, similar to lists, but they are **immutable**. They are generally used for heterogeneous data (different types) that belongs together, like coordinate pairs or database records.

Creating and Accessing Tuples

Tuples are created using parentheses (), though they are often optional. Accessing uses indexing and slicing, just like strings and lists.

Python

```
# Real-time Example: Storing a fixed GPS coordinate
```

```
gps_coord = (34.0522, -118.2437)
```

```
db_record = ("user_101", "active", "2025-01-15")
```

```
print(f"Latitude: {gps_coord[0]}")
```

```
print(f"Status: {db_record[1]}")
```

Packing and Unpacking Tuples

- **Packing:** Simply creating a tuple from a sequence of values.
- **Unpacking:** Assigning the elements of a tuple to individual variables. This is extremely common for function returns.

Python

```
# Packing (The tuple is created implicitly)
```

```
settings = 1024, True, "High"
```

```
# Unpacking (Commonly used for multiple assignment or iterating through key-value pairs)
```

```
max_res, is_enabled, quality = settings
```

```
print(f"Resolution: {max_res}, Enabled: {is_enabled}, Quality: {quality}")
```

```
# Real-time Example: Swapping variables (a classic tuple unpacking trick)
```

```
a = 10
```

```
b = 20
```

```
a, b = b, a # Swaps the values!
```

```
print(f"After swap, a: {a}, b: {b}") # a: 20, b: 10
```

Tuple Immutability

Tuples cannot be modified after creation. They are faster than lists and can be used as keys in dictionaries (unlike lists).

Python

```
status_code = (200, "OK")
```

```
# Attempting to change an item will result in a TypeError
```

```
# status_code[0] = 404 # TypeError: 'tuple' object does not support item assignment
```

Single-item tuple

A tuple with a single item must include a trailing comma (,) to distinguish it from a simple expression enclosed in parentheses.

Python

```
single_item_tuple = (10,) # This is a tuple
```

```
not_a_tuple = (10)    # This is just an integer (10)

print(f"Type of (10): {type(single_item_tuple)}") # <class 'tuple'>
print(f"Type of (10): {type(not_a_tuple)}")      # <class 'int'>
```

5.4 Dictionaries (Mutable Mapping)

Dictionaries are unordered collections of **Key-Value** pairs. They are **mutable**. They are used to map one item (the key) to another (the value), providing efficient lookup by key.

Creating and Accessing Dictionaries

Dictionaries are created using curly braces {} with colon-separated key-value pairs: {key1: value1, key2: value2}.

- **Accessing:** Use square brackets [key]. If the key is not found, it raises a KeyError.

```
Python
# Real-time Example: Storing configuration settings
config = {
    "host": "localhost",
    "port": 8080,
    "max_connections": 50,
    "debug_mode": True
}

# Accessing values
server_port = config["port"]
print(f"Server Port: {server_port}") # 8080

# Adding/Updating a key-value pair
config["debug_mode"] = False
config["timeout"] = 30
print(f"New Config: {config}")
```

Keys and Values

- **Keys:** Must be **immutable** and **unique**. Common key types are strings, numbers, and tuples. Lists and dictionaries cannot be keys.
- **Values:** Can be any data type (mutable or immutable).

Dictionary Methods

Method	Description	Real-time Use Case
.keys()	Returns a view object of all keys.	Iterating over all available configuration options.

Method	Description	Real-time Use Case
<code>.values()</code>	Returns a view object of all values.	Calculating the sum of all item prices.
<code>.items()</code>	Returns a view object of (key, value) tuples.	Iterating over key-value pairs for display or processing.
<code>.get(key, default)</code>	Returns the value for key if it exists, otherwise returns default (defaults to None). Recommended for safe access.	Safely accessing a setting that might not be in the config file.
<code>.pop(key)</code>	Removes the key and returns its value. Raises <code>KeyError</code> if key is not found.	Extracting and processing a task from a queue (using task ID as key).
<code>.update(other_dict)</code>	Merges another dictionary or key-value pairs into the current dictionary. Overwrites keys if they already exist.	Applying global default settings over user-specific settings.

Python

```
# Real-time Example: Processing user profile data
user_profile = {
    "username": "coder_xyz",
    "email": "c@example.com",
    "joined": "2024-10-01",
    "premium": False
}

# 1. Safe access (get)
status = user_profile.get("status", "No status provided")
print(f"Status: {status}") # No status provided

# 2. Get keys and iterate
print("User Fields:", list(user_profile.keys()))

# 3. Get key-value pairs (items)
print("Profile:")
for key, value in user_profile.items():
    print(f"- {key.capitalize()}: {value}")

# 4. Update the profile (update)
new_data = {"premium": True, "last_login": "2025-12-12"}
user_profile.update(new_data)
print(f"After Update: {user_profile}")
```

Dictionary Comprehensions (Basic Syntax)

A concise way to create dictionaries. Syntax: {key_expr: value_expr for item in iterable if condition}.

Python

Real-time Example: Creating a dictionary from two lists

```
product_ids = ["P101", "P102", "P103"]
```

```
product_names = ["Monitor", "Keyboard", "Mouse"]
```

Create ID -> Name mapping

```
product_map = {
```

```
    id: name
```

```
    for id, name in zip(product_ids, product_names)
```

```
}
```

```
print(f"Product Map: {product_map}") # {'P101': 'Monitor', 'P102': 'Keyboard', 'P103': 'Mouse'}
```

Example 2: Inverting a dictionary (if values are unique and immutable)

```
inverted_map = {
```

```
    v: k for k, v in product_map.items()
```

```
}
```

```
print(f"Inverted Map: {inverted_map}") # {'Monitor': 'P101', 'Keyboard': 'P102', 'Mouse': 'P103'}
```

5.5 Sets (Mutable, Unordered Collection of Unique Elements)

Sets are unordered collections of unique, immutable elements. They are highly optimized for membership testing and performing mathematical set operations.

Creating Sets (Differences from dictionary creation)

Sets are created using curly braces {} or the set() constructor.

Crucial Difference: An **empty dictionary** is created with {}, while an **empty set** is created with set().

Python

Real-time Example: Tracking unique visitors to a website

```
session_ids = {101, 102, 103, 101, 104} # Duplicate 101 is automatically removed
```

```
print(f"Unique Sessions: {session_ids}") # {101, 102, 103, 104} (Order is not guaranteed)
```

Creating an empty set

```
active_users = set()
```

Set Operations

Sets support mathematical set operations.

Operator	Method	Description	Real-time Use Case
`	`	.union()	All unique elements in both sets.
&	.intersection()	Elements common to both sets.	Finding users who bought <i>both</i> Product A <i>and</i> Product B.

Operator	Method	Description	Real-time Use Case
-	<code>.difference()</code>	Elements in the first set but not the second.	Finding users who logged in on Day 1 <i>but not</i> Day 2.
^	<code>.symmetric_difference()</code>	Elements in either set, but not in both.	Finding users who made a purchase <i>or</i> returned an item, but <i>not</i> both.

Python

Real-time Example: Analyzing customer product purchases

```
product_a_buyers = {"user_A", "user_B", "user_C"}
```

```
product_b_buyers = {"user_B", "user_D", "user_E"}
```

1. Union: All customers who bought at least one product

```
all_buyers = product_a_buyers | product_b_buyers
```

```
print(f"All Buyers (Union): {all_buyers}") # {'user_A', 'user_B', 'user_C', 'user_D', 'user_E'}
```

2. Intersection: Customers who bought both products

```
both_buyers = product_a_buyers & product_b_buyers
```

```
print(f"Both Buyers (Intersection): {both_buyers}") # {'user_B'}
```

3. Difference: Buyers of A but not B

```
a_only_buyers = product_a_buyers - product_b_buyers
```

```
print(f"A Only (Difference): {a_only_buyers}") # {'user_A', 'user_C'}
```

Set Methods

Method	Description	Real-time Use Case
<code>.add(item)</code>	Adds a single item to the set.	Adding a new session ID to a set of active sessions.
<code>.remove(item)</code>	Removes an item. Raises <code>KeyError</code> if the item is not present.	Removing a known processed ID.
<code>.discard(item)</code>	Removes an item if present. Does not raise an error if the item is not present.	Safely removing a temporary file name that may or may not exist in a list of files to clean up.

Python

Real-time Example: Managing temporary file permissions

```
temp_files_to_clean = set()
```

Add new files

```
temp_files_to_clean.add("temp_1.log")
```

```
temp_files_to_clean.add("temp_2.bak")
```

Process and remove known file (remove)

```
temp_files_to_clean.remove("temp_1.log")

# Attempt to remove a file that might not be there (discard - safe)
temp_files_to_clean.discard("temp_99.tmp") # No error raised

print(f"Remaining files to clean: {temp_files_to_clean}") # {'temp_2.bak'}
```

frozenset (Immutable version of set)

A frozenset is an immutable set. Once created, you cannot add or remove elements. This makes it hashable, so it can be used as a **key in a dictionary** or as an **element in another set**.

Python

```
# Real-time Example: Using a fixed set of permissions as a dictionary key
# This can't be done with a regular mutable set.
permissions = frozenset(["read", "write"])
```

```
access_levels = {
    permissions: "Administrator",
    frozenset(["read"]): "Guest"
}
```

```
print(f"Access level for {list(permissions)}: {access_levels[permissions]}")
```

6. Modules and Packages

6.1 Modules

What is a Module?

A module is essentially a **single Python file** (with the .py extension) that contains definitions, statements, and functions. Modules allow you to logically organize your Python code, making it reusable and easier to maintain.

Real-time Analogy: Think of a module as a specialized **tool kit** (e.g., a math tool kit or a file-handling tool kit). When you need to perform a specific task, you import the relevant tool kit instead of rewriting the tools yourself.

Creating a Module

Creating a module is as simple as saving code in a .py file.

Example: data_processor.py Let's create a module for processing sales data.

Python

```
# --- File: data_processor.py ---
```

```
# Global variable for configuration
TAX_RATE = 0.05
```